

SweetLab

Guía Visual y Conceptual del Proyecto
Full-Stack Web Development

Angular 18

Node.js + Express

MongoDB Atlas

JWT Auth

REST API

AUTORA

Mirel Volcan Calderon (MirelSIG)

GUÍA

Mirel Volcan Calderon

FECHA DE ENTREGA

26/05/2026

PRODUCCIÓN

sweetlab-kf27.onrender.com

REPOSITORIO

github.com/MirelSIG/SweetLab

TIPO

Proyecto de Práctica Bootcamp

Índice de Contenidos

Esta guía está pensada para reclutadores y personas no técnicas que quieren entender qué construyó Mirel y por qué importa.

01 ¿Qué es SweetLab?

Visión general del proyecto

02 Arquitectura del Sistema

Mapa mental + diagrama corregido

03 Mapa Mental: Tecnologías

Angular 18, Node.js, MongoDB

04 UML de Clases

Modelos de datos del sistema

05 UML de Secuencia: CRUD

Flujo completo de operaciones

06 Autenticación JWT

Diagrama de flujo de seguridad

07 Estructura de Archivos

Mapa del proyecto

08 Endpoints de la API

Tabla de referencia completa

09 Interfaz de Usuario

Pantallas y flujos

10 Glosario Técnico

Términos explicados sin jerga

01 ¿Qué es SweetLab?

SweetLab es una **aplicación web completa** construida como proyecto de práctica de bootcamp. Funciona como un catálogo interactivo de recetas de cocina donde cualquier persona puede explorar recetas, y los administradores pueden crear, editar y eliminar contenido.

🔗 ¿Por qué importa esto para un reclutador?

SweetLab no es un ejercicio simple. Es una aplicación con **seguridad real** (tokens JWT), **base de datos en la nube** (MongoDB Atlas) y **despliegue en producción** (accesible desde internet). Mirel construyó exactamente lo que haría en un trabajo real.

Lo que puede hacer un usuario

👁 Vista Pública

Cualquier persona puede explorar el catálogo, buscar recetas por nombre o ingrediente y ver detalles completos sin necesidad de cuenta.

🔑 Panel Admin

El administrador inicia sesión con credenciales seguras y puede crear recetas nuevas, editarlas (incluida imagen) y eliminarlas de la base de datos.

El recorrido de aprendizaje de Mirel

Este proyecto fue construido progresivamente, dominando cada capa antes de pasar a la siguiente:

Paso 1 **Frontend estático** — Angular mostrando recetas ficticias desde un archivo JSON local

Paso 2 **Backend API** — Servidor Node.js/Express con MongoDB para guardar recetas reales

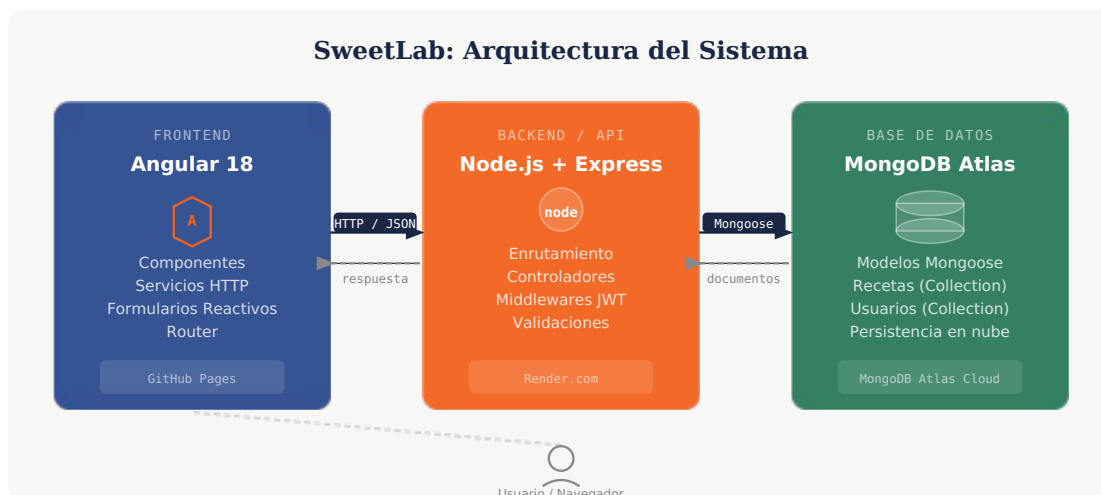
Paso 3 **Integración CRUD** — Angular conectado a la API: crear, leer, actualizar y eliminar recetas

Paso 4 **Seguridad JWT** — Sistema de autenticación con tokens, roles y bloqueo por intentos fallidos

Paso 5 **Producción** — Despliegue en Render + GitHub Pages, accesible desde cualquier navegador del mundo

02 Arquitectura del Sistema

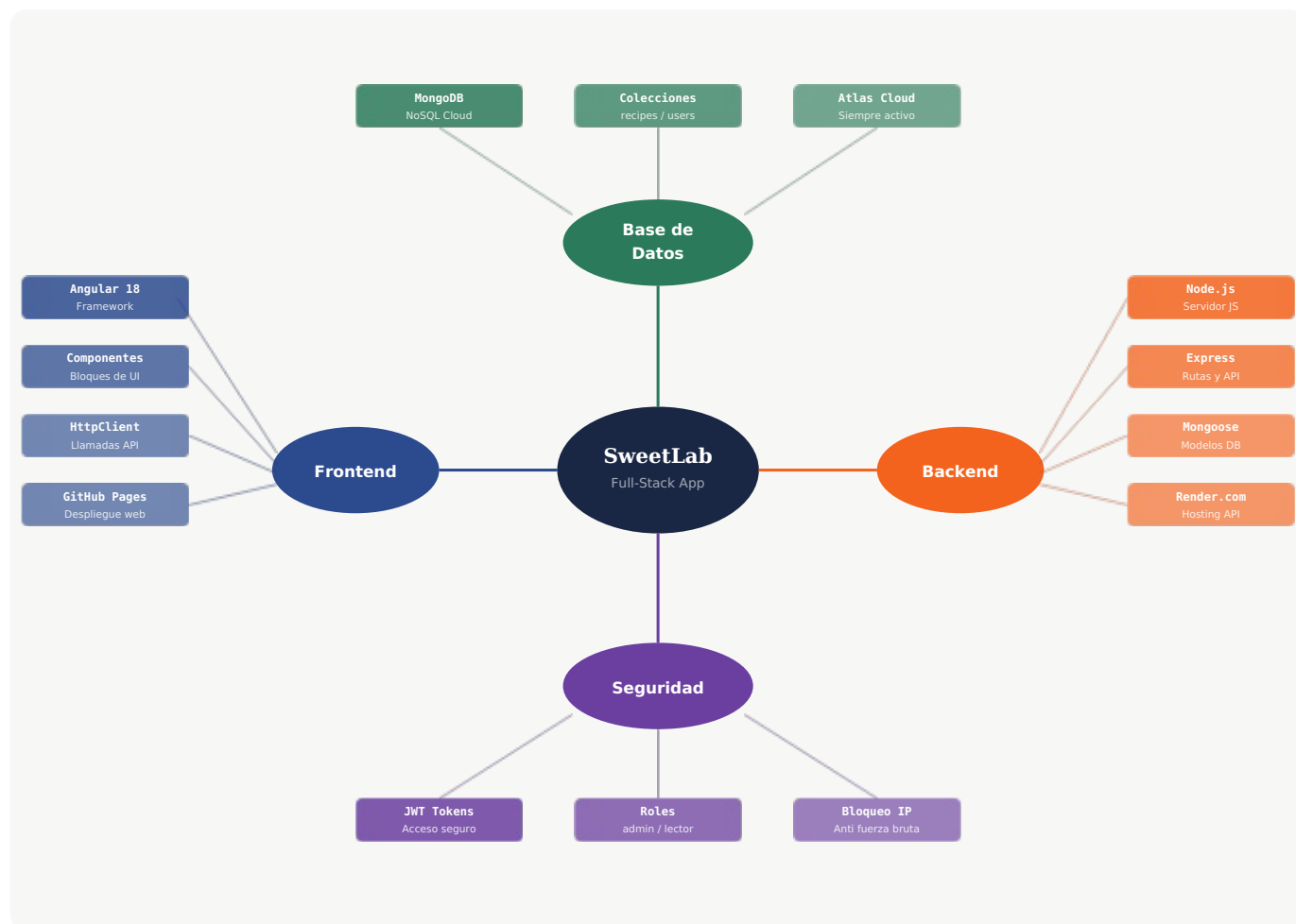
Una aplicación web moderna se compone de tres capas separadas que se comunican entre sí. Aquí la arquitectura real de SweetLab con las tecnologías correctas:



Arquitectura de tres capas de SweetLab — Angular 18 · Node.js/Express · MongoDB Atlas

03 Mapa Mental: Stack Tecnológico

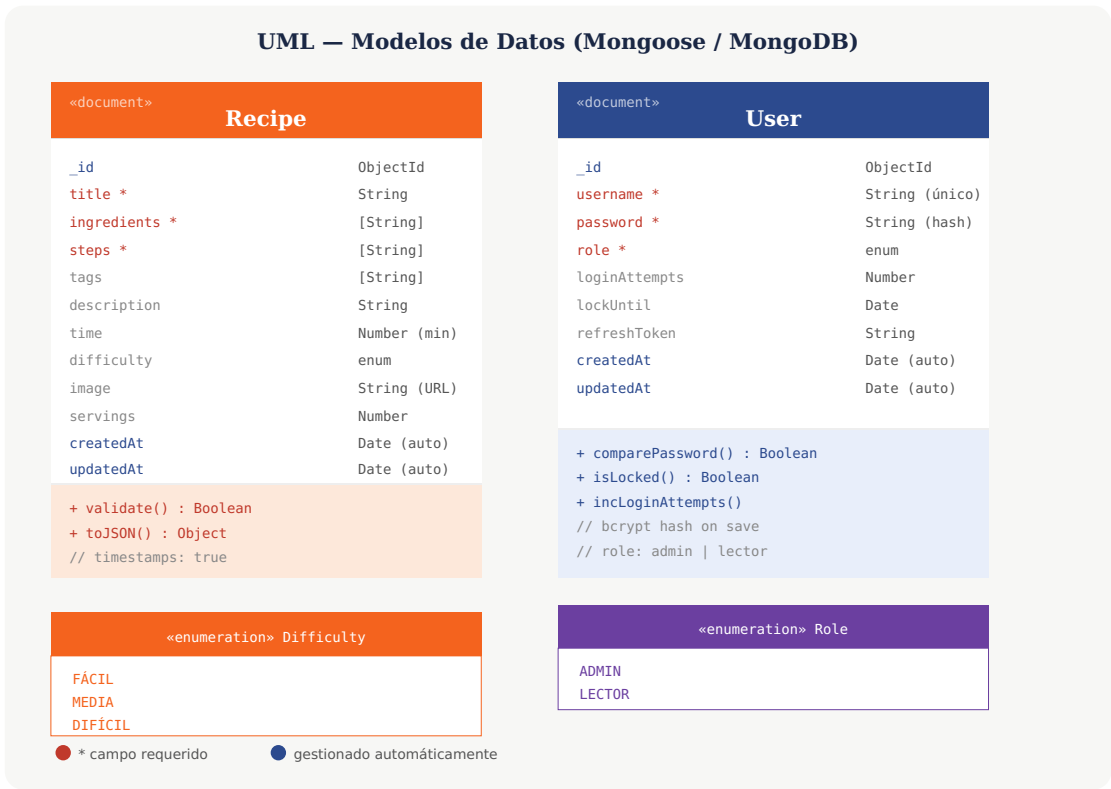
Cada tecnología en SweetLab cumple un rol específico. Este mapa mental muestra cómo se relacionan todas las piezas del sistema:



Mapa mental del ecosistema tecnológico de SweetLab

04 Diagrama UML de Clases

Los **diagramas UML de clases** muestran los "moldes" de datos que usa el sistema. Piensa en ellos como las fichas de un catálogo: cada ficha tiene campos con información. Aquí están los modelos de SweetLab:



Modelos de datos de SweetLab — Recipe y User con sus atributos y métodos

🔗 ¿Qué es un campo "requerido"?

Los campos marcados con * son obligatorios: si intentas guardar una receta sin título o sin ingredientes, el sistema devuelve un error claro. Esto es validación de datos, una práctica fundamental en el desarrollo profesional.

05 UML de Secuencia — Flujo CRUD

Un **diagrama de secuencia** muestra quién habla con quién y en qué orden cuando ocurre una acción. Aquí el flujo completo cuando un admin crea una nueva receta:

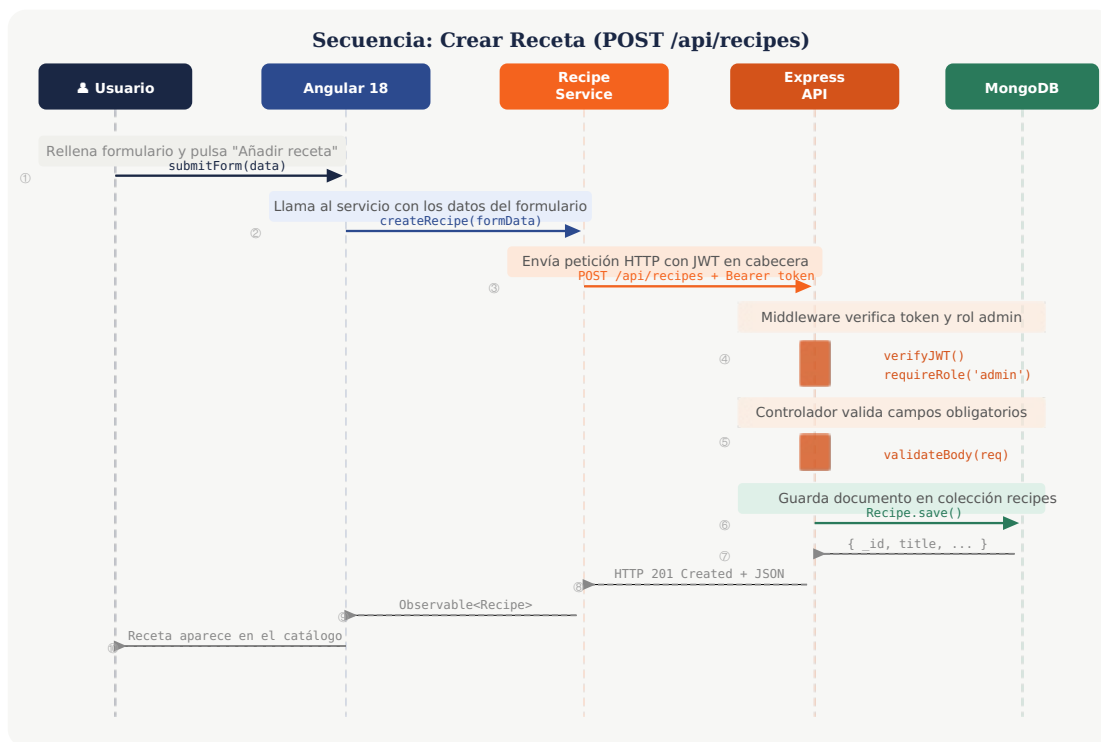
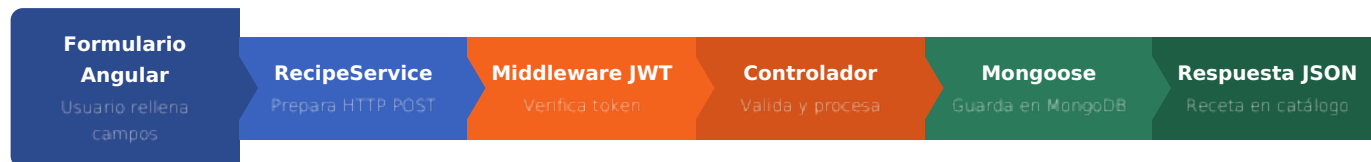


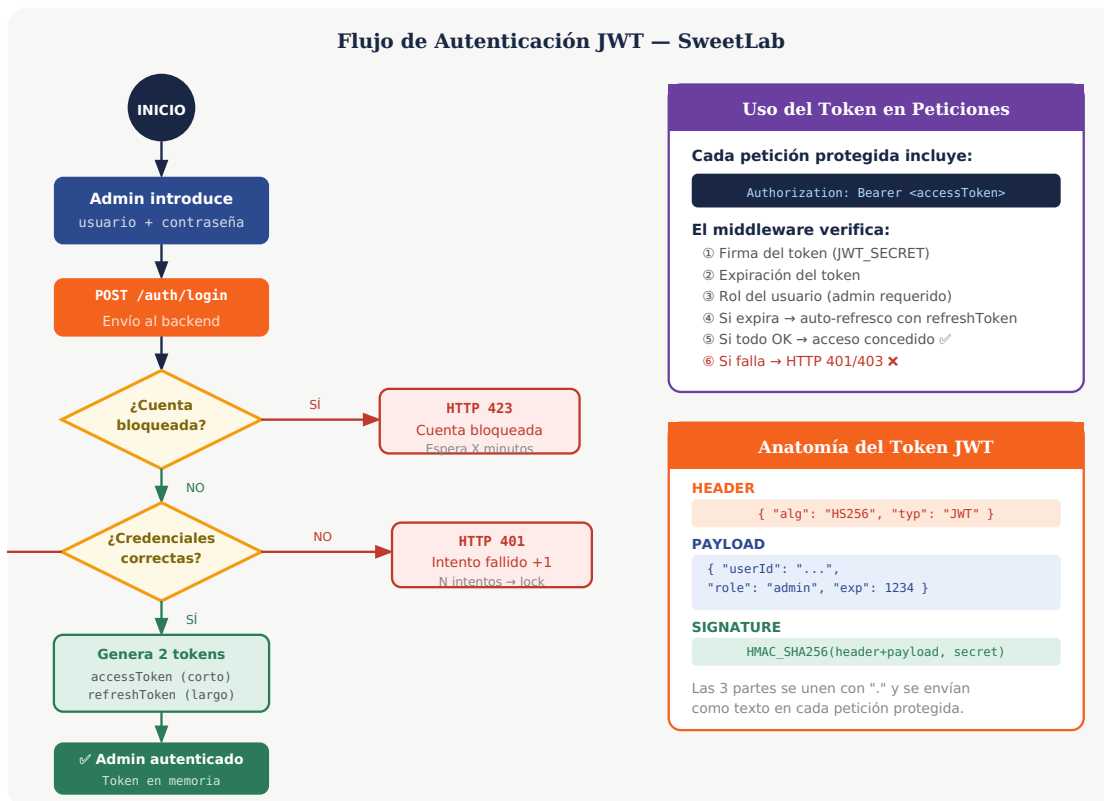
Diagrama UML de secuencia — Flujo completo de creación de receta con autenticación

La ruta completa de una receta



06 Diagrama de Flujo — Autenticación JWT

JWT (JSON Web Token) es el sistema de seguridad que usa SweetLab. Funciona como una **pulsera de acceso VIP**: el servidor la entrega al entrar y el usuario la presenta en cada puerta restringida.



Flujo de autenticación JWT con manejo de bloqueos, tokens y renovación automática

🔒 Seguridad real implementada

SweetLab incluye **bloqueo temporal por IP y usuario** tras varios intentos fallidos de login. Esto protege contra ataques de fuerza bruta, una amenaza real en aplicaciones web. No es un detalle menor: es una característica de seguridad de nivel profesional.

07 Estructura del Proyecto

El proyecto está organizado en tres grandes carpetas, cada una con una responsabilidad clara:

frontend/

```
src/  
├─ main.ts  
Punto de entrada  
├─ app/  
  │ └─ app.component.ts  
  Lógica principal  
  │ └─ recipe.service.ts  
  Llamadas API  
  │ └─ recipe.model.ts  
  Tipo de datos  
└─ assets/  
  └─ recipes.json  
  Datos de ejemplo
```

backend/

```
src/  
├─ server.js  
Arranque y MongoDB  
├─ app.js  
Config middleware  
├─ routes/  
  │ └─ recipeRoutes.js  
  │ └─ authRoutes.js  
├─ controllers/  
  │ └─ recipeController.js  
├─ models/  
  │ └─ Recipe.js  
  │ └─ User.js  
└─ middleware/  
  └─ auth.js
```

docs/

```
├─ deploy-render.md  
Guía de despliegue  
├─ github-pages-setup.md  
Config GitHub Pages  
└─ recipes-example.json  
Datos de prueba
```

📄 README.md

Documentación completa del proyecto con instrucciones paso a paso

🔗 Separación de responsabilidades

Tener el frontend, backend y documentación en carpetas separadas no es solo orden: es una **convención profesional** llamada "separación de capas". Facilita el trabajo en equipo y el mantenimiento a largo plazo.

08 Endpoints de la API REST

Una **API REST** es como un menú de acciones disponibles: cada "endpoint" es una URL específica que realiza una operación concreta sobre los datos. SweetLab tiene dos grupos de endpoints:

Autenticación

MÉTODO	RUTA	ACCIÓN	AUTENTICACIÓN
POST	/auth/login	Iniciar sesión y obtener tokens	✕ Pública
POST	/auth/refresh	Renovar el access token expirado	🔑 refreshToken
POST	/auth/logout	Cerrar sesión y limpiar tokens	🔑 JWT requerido

Recetas (CRUD completo)

MÉTODO	RUTA	ACCIÓN	PERMISOS
GET	/api/recipes	Listar todas las recetas	🌐 Pública
GET	/api/recipes/:id	Ver detalle de una receta por ID	🌐 Pública
POST	/api/recipes	Crear receta nueva (con imagen)	🔑 Solo admin
PUT	/api/recipes/:id	Actualizar receta existente	🔑 Solo admin
DELETE	/api/recipes/:id	Eliminar receta por ID	🔑 Solo admin

Respuestas de error estándar

400	Body vacío o campos faltantes
401	Token inválido o credenciales incorrectas
403	Sin permisos de admin
404	Receta no encontrada por ese ID

✔ Validaciones pensadas para aprender

Mirel implementó mensajes de error claros y específicos. Cuando una petición falla, el mensaje explica exactamente qué salió mal: "El campo title es obligatorio", "El ID proporcionado no es válido". Esto es una práctica de desarrollo profesional que facilita el debugging.

09 Interfaz de Usuario y Flujos

SweetLab tiene dos modos de uso claramente diferenciados según el rol del visitante:

VISTA PÚBLICA (SIN LOGIN)

SL SWEETLAB

Acceso admin

Recetas, edición y experimentación

Explora, crea y organiza recetas.

🔍 Buscar receta...

Tarta de Vainilla
#postre #festivo

Brownies Clásicos
#chocolate #rápido

Cualquier visitante puede navegar y buscar recetas sin necesidad de cuenta.

PANEL ADMINISTRADOR (CON JWT)

SL SWEETLAB

admin conectado Cerrar sesión

+ Añadir receta

Editar

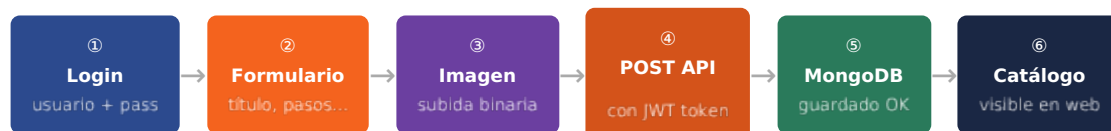
Eliminar

RECETA SELECCIONADA

Tarta Lunar de Vainilla
50 min · 8 porciones · Media
Ingredientes: 2 tazas harina, azúcar, huevos...

El admin ve los controles de gestión inyectados dinámicamente tras el login.

Flujo de creación de receta (Admin)



Términos clave del proyecto explicados sin jerga innecesaria, para que cualquier persona pueda entender de qué trata cada tecnología o concepto:

API REST

Sistema de comunicación entre el frontend y el backend mediante URLs. Como un menú de restaurante: pides algo por una URL específica y recibes exactamente lo que pediste en formato JSON.

Angular 18

Framework de Google para construir interfaces web. Permite crear aplicaciones interactivas divididas en "componentes" reutilizables. Cada componente tiene su lógica, su plantilla visual y su estilo.

Node.js

Entorno que permite ejecutar JavaScript en el servidor (no solo en el navegador). Es el motor sobre el que corre el backend de SweetLab.

Express

Librería de Node.js que facilita crear servidores web y APIs. Define qué pasa cuando alguien visita cada URL del backend.

MongoDB

Base de datos que guarda información en documentos JSON (no en tablas como Excel). Cada receta es un "documento" dentro de una "colección". MongoDB Atlas es su versión en la nube.

Mongoose

Librería que conecta Node.js con MongoDB. Permite definir la "forma" exacta que deben tener los datos (modelo) antes de guardarlos.

JWT — JSON Web Token

Credencial digital cifrada que el servidor entrega al hacer login. El usuario la incluye en cada petición protegida, como una pulsera de acceso VIP que el servidor puede verificar sin consultar la base de datos.

CRUD

Las 4 operaciones básicas sobre datos: **C**reate (crear), **R**ead (leer), **U**pdate (actualizar), **D**elete (eliminar). SweetLab las implementa todas para las recetas.

Componente (Angular)

Bloque de interfaz reutilizable que combina HTML, CSS y TypeScript en una unidad. Como un "widget" que se puede colocar en distintas partes de la página.

Servicio (Angular)

Clase que centraliza tareas comunes como hacer llamadas HTTP a la API. Separa la lógica de datos de la lógica visual, siguiendo buenas prácticas de diseño.

Middleware

Función que se ejecuta automáticamente entre que llega una petición y que el servidor la procesa. En SweetLab, verifica el JWT antes de dejar entrar a las rutas protegidas.

Formulario Reactivo

Formulario web gestionado completamente desde el código TypeScript (no desde el HTML). Permite validaciones complejas y control total sobre los datos antes de enviarlos.

GitHub Pages

Servicio gratuito de GitHub que publica el frontend Angular como una web accesible desde cualquier navegador. El despliegue es automático con cada push al repositorio.

Render.com

Plataforma en la nube que aloja el backend Node.js/Express en producción. El servidor está siempre activo y accesible desde internet, sin necesidad de un ordenador encendido.

FormData

Objeto de JavaScript que permite enviar ficheros binarios (imágenes) junto con datos de texto en una misma petición HTTP. SweetLab lo usa para subir la imagen de cada receta.

Observable (RxJS)

Mecanismo de Angular para manejar datos que llegan de forma asíncrona (como una respuesta del servidor que tarda unos segundos). Permite "suscribirse" y reaccionar cuando los datos llegan.

¿Qué demuestra este proyecto?

- ✓ Arquitectura full-stack real (3 capas)
- ✓ Autenticación y autorización JWT
- ✓ CRUD completo con validaciones
- ✓ Despliegue en producción (2 servicios)
- ✓ Código limpio y documentado
- ✓ Tests automatizados con cobertura
- ✓ Gestión de imágenes (FormData)
- ✓ Seguridad anti fuerza bruta

Guía y proyecto elaborados por **Mirel Volcan Calderon** · Peñascal F5 · Mayo 2026